

RISC-V 汇编程序编程练习

2021 年 3 月

一、实验介绍

前面的实验介绍了 RISC-V 工具链、GDB 调试器以及体系结构仿真器 QEMU 的用法。本次实验将进行 RISC-V 汇编语言的编程练习，并使用前面实验中介绍的工具来编译、调试和运行该 RISC-V 程序。

二、RISC-V 汇编规范

在 RISC-V Spec 文档中有以下的表格（[riscv-spec-20191213: Chapter 25](#)）:

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5	t0	Temporary/alternate link register	Caller
x6-7	t1-2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller
f0-7	ft0-7	FP temporaries	Caller
f8-9	fs0-1	FP saved registers	Callee
f10-11	fa0-1	FP arguments/return values	Caller
f12-17	fa2-7	FP arguments	Caller
f18-27	fs2-11	FP saved registers	Callee
f28-31	ft8-11	FP temporaries	Caller

Table 25.1: Assembler mnemonics for RISC-V integer and floating-point registers, and their role in the first standard calling convention.

这张表格包含了 RISC-V 标准定义的寄存器（x0-x31、f0-f31）在汇编语言中的别名、约定用途、函数调用时的保存方式。该表是编写 RISC-V 汇编程序时最重要的规范之一，几乎所有的通用编译器都遵照该规范来生成可执行文件。

2.1 寄存器别名和约定用途

在汇编中，每一个 RISC-V 寄存器（`x0-x31`、`f0-f31`）都拥有一个别名（ABI Name），它们在使用时是等价的。但是由于别名通常暗示了这个寄存器的用途，使用别名的汇编代码可读性更佳，因此更加推荐使用。例如，以下的两条汇编指令是等价的：

```
add x10,x10,x0
add a0,a0,zero
```

每个寄存器都有其约定用途，为了保持编译器的兼容性，编程时总是应当遵循这个约定用途。对于本课程而言，比较重要的寄存器如下（这里仅列出其别名）：

- **zero**: 该寄存器在硬件设计时应当被硬接线到 0，任何读取该寄存器的操作均得到结果 0，而任何写入该寄存器的操作都是无效的。
- **ra**: 返回地址。当过程调用发生时（例如发生子函数的调用），应当先向 **ra** 寄存器写入程序的返回地址，然后才能转移到调用入口。当过程调用通过 **ret** 一类的指令返回时，也要通过读取 **ra** 寄存器的内容并赋值给 **pc** 寄存器，硬件才能知道程序应当返回到何处。

子程序的返回指令 **ret** 实际上是一条伪指令，它等价的基础指令为：

```
jalr x0,x1,0
```

jalr 指令定义的操作如下：

语法:	<code>jalr rd, rs1, imm12</code>
操作:	$\text{next pc} \leftarrow (\text{rs1} + \text{sign_extend}(\text{imm12})) \& 64'\text{hfffffffffffffe}$ $\text{rd} \leftarrow \text{current pc} + 4$

对应起来看，`jalr x0,x1,0` 的 `rd=x0`, `rs1=x1`, `imm12=0`。由此可以看到，**ret** 指令正是把 **x1** 寄存器（即 **ra**）的值赋给了 **pc** 寄存器，从而函数可以正确返回其调用者的位置。此外，对 **x0** 的赋值永远是无效的，也就相当于不执行 `rd←current pc + 4` 这一步。

- **sp**: 堆栈指针。维护运行时栈的栈顶位置。在我们所使用的硬件中，堆栈在过程调用中向低地址生长，因此 **sp** 指向当前堆栈的最小地址。
- **s0/fp**: 栈帧指针。**fp** 与 **sp** 的具体区别和用途超出了本讲义的范畴，可以参考 2.3.1 节简要地理解它的用法。
- **a0-a7**: 函数参数。当父函数调用子函数 `func(arg1,arg2,arg3...)` 时，父函数应当将传递的参数 `arg1`、`arg2`、`arg3...` 顺次存放在寄存器 **a0**、**a1**、**a2...** 中，如果函数的参数超过了 8 个，则剩下的参数应当存放在栈中。当子函数 `func` 执行时，它将默认从 **a0** 开始的寄存器取出相应的参数。
- **a0**: 函数返回值。过程调用返回时，约定把返回值存放在 **a0** 中。

2.2 过程调用的寄存器保存规则

当过程调用发生时，父函数 **P** 可能希望子函数 **Q** 除了返回值以外，不要修改任何其他寄存器里面的值，这样父函数 **P** 就可以在 **Q** 返回后轻松地继续执行后续计算。但这是不现实的，因为通用寄存器只有 32 个，所有的过程调用都只能共享这些通用寄存器来执行计算，因此，子函数 **Q** 总是有可能会覆盖父函数 **P** 使用的寄存器。

为了保持程序具有正确的功能，汇编程序编程规范的最后列“Saver”规定了寄存器的保存规则。其中：

- **Caller** 代表调用者保存寄存器，父函数 P 如果把临时数据存放在了 **Caller** 类型的寄存器中，则在调用子函数 Q 之前，P 有义务把自己的临时数据放到栈中，Q 在执行过程中可以随意地修改这些寄存器中的值。在 Q 返回后，P 应当自行从栈中恢复那些数据。
- **Callee** 代表被调用者保存寄存器，父函数 P 可以期望所有 **Callee** 类型的寄存器值在调用 Q 之前和调用 Q 之后是保持不变的。如果 Q 想要使用这些寄存器，则它必须首先把寄存器中的原始值保存到栈中，并在 Q 返回之前把栈中的值恢复到寄存器中。从 P 的视角来看，结果就好像是 Q 从未使用过这些寄存器一样。

汇编编程时，遵循寄存器保存规则至关重要，它是程序正确性的重要保障。

2.3 实例介绍

在《GDB 补充介绍》实验中，使用了一个示例程序源码 `example.c`，其中包含两个求最大公约数的 C 函数 `gcd_1` 和 `gcd_2`，该示例程序在 GDB 调试实验中通过 RISC-V GCC 工具链编译成了二进制文件 `example`。本文使用 RISC-V 工具链生成了 `gcd_1` 的汇编代码，将其整理和修改成为了本次实验的示例文件 `gcd_1.s`，该文件位于文件夹 `lab4_assembler/sample` 下。本节接下来的内容将会以递归函数 `gcd_1` 的汇编代码为例，介绍 RISC-V 汇编规范的具体使用方法。

`gcd_1` 的 C 源码为：

```
int gcd_1(int a, int b) //辗转相除法
{
    if(a==0) return b;
    else if(b==0) return a;
    else if(a>b) return gcd_1(a%b,b);
    else return gcd_1(b%a,a);
}
```

`gcd_1.s` 通过 6 个标号（`start`、`part1`、`part2`、`part3`、`part4`、`end`）将汇编程序分成了六个部分，这六个部分各自的功能如下：

- **start**: 函数起点，其后的几行代码执行了运行时栈的初始化工作
- **part1**: 对应 C 代码 `if(a==0) return b;`
- **part2**: 对应 C 代码 `else if(b==0) return a;`
- **part3**: 对应 C 代码 `else if(a>b) return gcd_1(a%b,b);`
- **part4**: 对应 C 代码 `else return gcd_1(b%a,a);`
- **end**: 设置返回值，恢复寄存器现场，函数返回

接下来对其每一个部分的汇编代码作介绍。

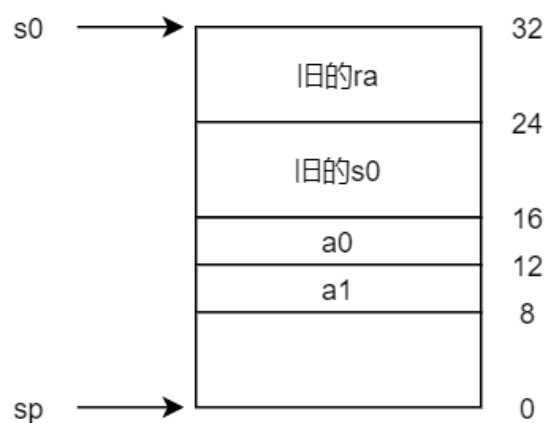
2.3.1 start

该标号直到 `part_1` 为止的代码为：

```
start:  addi    sp, sp, -32
        sd      ra, 24(sp)
        sd      s0, 16(sp)
        addi    s0, sp, 32
        mv      a5, a0
        mv      a4, a1
        sw      a5, -20(s0)
        mv      a5, a4
        sw      a5, -24(s0)
```

这段代码执行了运行时栈的初始化工作。`gcd_1()` 接受两个整型参数，按照汇编规范的约定这两个参数可以从 `a0, a1` 两个寄存器中读到。

代码首先将栈顶向低地址移动 32 个字节，将 `ra`、`s0` 寄存器的原始值存进栈中，然后把输入参数 `a0` 和 `a1` 的值也存入栈中。初始化完成后，这个栈的内容如下图所示：



注：图中的地址为 10 进制，它只是为了表示方便而标注的相对栈顶 `sp` 而言的相对地址，不是绝对地址。

注意到在实验所用的机器上，运行时栈的栈顶是向地址减小的方向生长的，因此代码中初始化时对 `sp` 做的是减法，即 `sp` 总是位于当前栈的最小地址。详细地说明为什么需要引入 `s0` 这个额外的指针超出了本文的范畴，这里可以并不严谨地简单理解为：`s0` 就是 `sp` 一开始的值，它指向了当前栈帧的最大地址，`sp` 和 `s0` 共同构成了一个运行时栈的首尾。

Question: 为什么程序要在栈中保存图示的 4 个值？

A:

- 对于 `ra`，如 2.1-2.2 节所叙，`ra` 存储了函数的返回地址。当函数 `P` 调用子函数 `Q` 时，`P` 先把 `Q` 的返回地址写进 `ra`，记为 `ra(P)`，然后调用 `Q`。`Q` 执行完成后，正是通过 `ra(P)` 的值来决定应该返回到什么地址的。但是，函数 `Q` 自己也可能存在子函数调用，假设 `Q` 调用 `R`，则 `Q` 也应该先把 `R` 的返回地址写进 `ra`，记为 `ra(Q)`，这样 `R` 执行完成后才能知道该返回到何处。但显然，`ra(Q)` 的写入操作会覆盖 `ra(P)` 的值，因此 `Q` 不得不在一开始就先把 `ra(P)` 的储存在栈里。
- 对于 `s0`，这是一个被调用者保存寄存器，而程序后续即改变了 `s0` 的值，所以将其旧值

保存起来。

- 对于 `a0` 和 `a1`，它们存放的是本次 `gcd_1` 的两个输入参数，且在计算过程中这两个参数会被反复使用。由于 `gcd_1` 存在子函数调用，`a0` 和 `a1` 作为调用者保存寄存器，不能够期待在子函数调用前后 `a0` 和 `a1` 依然维持原值，因此只能把它们存放在栈中，并在每次使用时从栈中读取。

2.3.2 part1

该标号直到 `part_2` 为止的代码为：

```
part1:  lw      a5, -20(s0)
        sext.w  a5, a5
        bnez    a5, part2
        lw      a5, -24(s0)
        j       end
```

这段对应 C 代码 `if(a==0) return b;`

它从栈中取出 `a` 的值，并通过 `bnez` 指令进行条件跳转，如果 `a==0`，则将 `b` 的值赋给 `a5` 寄存器的值（为什么是 `a5` 见 2.3.6），并跳转到 `end` 标号，否则跳转到 `part2` 标号（即下一条 `else if` 语句）。

2.3.3 part2

该标号直到 `part_3` 为止的代码为：

```
part2:  lw      a5, -24(s0)
        sext.w  a5, a5
        bnez    a5, part3
        lw      a5, -20(s0)
        j       end
```

这段对应 C 代码 `else if(b==0) return a;`

其结构和 `part1` 基本完全一样，见 2.3.2 节，不再赘述。

2.3.4 part3

该标号直到 `part_4` 为止的代码为：

```
part3:  lw      a4, -20(s0)
        lw      a5, -24(s0)
        sext.w  a4, a4
        sext.w  a5, a5
        bge     a5, a4, part4
        lw      a4, -20(s0)
```

```

lw      a5, -24(s0)
remw    a5, a4, a5
sxt.w   a5, a5
lw      a4, -24(s0)
mv      a1, a4
mv      a0, a5
jal     ra, gcd_1
mv      a5, a0
j       end

```

这段对应 C 代码 `else if(a>b) return gcd_1(a%b,b);`

它取出栈中的两个参数 `a` 和 `b` 的值，放进寄存器 `a4` 和 `a5`，通过 `bge` 指令比较它们的大小并执行条件跳转，当 `a5 ≥ a4`（即 `b ≥ a`）时，跳转到 `part4` 标号。否则（即 `a > b`），通过 `remw` 指令计算出 `a%b` 的值，再将 `a%b` 和 `b` 的值分别存放在 `a0` 和 `a1` 寄存器中（这正是遵循 RISC-V 汇编编程规范的参数传递寄存器），用 `jal` 指令发生递归调用。

`jal` 指令定义的操作为：

语法: `jal rd, label`
 操作: $next\ pc \leftarrow current\ pc + sign_extend(imm20 \ll 1)$
 $rd \leftarrow current\ pc + 4$

`jal` 的递归调用结束后，子程序的返回值按照规范存放在 `a0` 中，同时控制将会返回到 `jal` 的下一行指令，即 `mv a5, a0`。这表明程序把递归调用的返回值复制到了 `a5` 寄存器（为什么是 `a5` 见 2.3.6），然后跳转到 `end` 标号。

2.3.5 part4

该标号直到 `part_5` 为止的代码为：

```

part4: lw      a4, -24(s0)
        lw      a5, -20(s0)
        remw    a5, a4, a5
        sxt.w   a5, a5
        lw      a4, -20(s0)
        mv      a1, a4
        mv      a0, a5
        jal     ra, gcd_1
        mv      a5, a0

```

这段对应 C 代码 `else return gcd_1(b%a,a);`

它和 `part3` 比较像，取出栈中的两个参数 `a` 和 `b` 的值，放进寄存器 `a4` 和 `a5`，用 `remw` 指令计算出 `b%a` 的值，再将 `b%a` 和 `a` 的值分别放在 `a0` 和 `a1` 寄存器中，用 `jal` 指令发生递归调用。

代码后面的步骤和 `part3` 相同。

2.3.6 end

该标号直到程序结束的代码为：

```
end:    mv      a0,a5
        ld      ra,24(sp)
        ld      s0,16(sp)
        addi    sp,sp,32
        ret
```

这是程序即将返回前做的准备工作。首先，前面 **part1-part4** 的每个部分都把最终的计算结果暂存在了 **a5** 寄存器当中，但是按照 **RISC-V** 汇编编程规范最终的返回值要放在 **a0** 当中，因此 **mv a0,a5** 指令把结果转移进 **a0**，接下来从栈中恢复 **ra** 和 **s0** 的值，并把栈顶 **sp** 恢复到最开始的值。此后执行 **ret** 指令（此时 **ra** 已经从栈中恢复了，因此可以正确返回到它的父函数中）返回。

至此，**gcd_1** 函数的一次调用结束。

三、 编程练习

编程练习位于文件夹 **lab4_assembler/exercise** 下，该路径包含以下文件：

```
-lab4_assembler
  -exercise
    -mul.s      # 待用户补全的文件
    -fac.s      # 待用户补全的文件
    -main.c     # 测试用主函数
    -Makefile   # 编译脚本
```

其中 **mul.s** 和 **fac.s** 是本次练习需要完成的两个文件，其中必要的首尾指令已经写好。

3.1 编程要求

3.1.1 练习 1: mul.s

要求：仅使用 **RV64I** 整型指令集，实现两个 32 位有符号 **int** 型变量的乘法（不可以使用 **RV64M** 中的乘除法指令）。函数已在 **mul.s** 中被命名为 **mymul**。

函数原型：**int mymul(int,int)**；即接收两个 32 位有符号 **int** 作为输入，输出它们的乘积，仍以 **int** 表示。

3.1.2 练习 2: fac.s

要求：仅使用 RV64I 整型指令集，以递归函数的形式实现自然数的阶乘函数。其中，乘法操作应当调用练习 1 当中的 `mymul` 函数（直接把 `mymul` 视为一个标号即可）。函数已经在 `fac.s` 中被命名为 `myfac`。

函数原型：`int myfac(int);` 即接受一个非负的 `int` 型参数作为输入，输出它的阶乘，仍以 `int` 表示。（不需要考虑输入为负数的情况）

注：定义 0 的阶乘为 1

3.2 测试说明

`lab4_assembler/exercise` 文件夹下已经写好了测试程序 `main.c` 以及编译脚本 `Makefile`，直接在该目录下执行以下命令：

```
$ make
```

即可完成编译。如果要清除之前的编译结果，可以执行命令 `make clean`。如果编译成功，则该目录下会生成一个二进制文件 `test`。

此后，可以使用 `qemu` 运行二进制文件 `test` 并验证结果：

```
$ qemu-riscv64 test
```

如果程序编写正确，打印出的结果应当如下图所示：

```
student@ubuntu:~/CA_Lab/assembler/exercise$ qemu-riscv64 test
Test for mymul:
0*5=0
(-3)*0=0
3*5=15
(-3)*5=-15
3*(-5)=-15
(-3)*(-5)=15

Test for myfac:
0!=1
1!=1
2!=2
3!=6
4!=24
5!=120
6!=720
7!=5040
8!=40320
9!=362880
```

结果与上图吻合即表明测试通过。

提示：善用调试

提供的 `Makefile` 默认情况下已经为二进制程序 `test` 引入了 `gdb` 调试支持。编程中难免出现初次编写的程序结果有误的情况，此时使用实验 2 中介绍的 `gdb` 调试工具对 `test` 程序进行调试是一个很好的查错方法。

`gdb` 工具支持汇编代码层面的单步调试，`next` 和 `step`（缩写为 `n` 和 `s`）是 C 语言级的单

步调式命令，`ni` 和 `si` 则是相应的汇编语言级的单步调式命令。

此外，`gdb` 调试中可以使用 `print/display` 命令来打印寄存器的值，例如：`p $a1` 可以打印寄存器 `a1` 的值。`info registers` 则可以打印出所有寄存器的当前值。如下图所示：

```
(gdb) p $a0
$8 = 0
(gdb) p $a1
$9 = 5
(gdb) p $ra
$10 = (void (*)(void)) 0x101e8 <main+26>
(gdb) i registers
ra          0x101e8  0x101e8 <main+26>
sp          0x40007ffdd0 0x40007ffdd0
gp          0x1eb90  0x1eb90 <__malloc_av_+248>
tp          0x0      0x0
t0          0x15     21
t1          0x7      7
t2          0x16     22
fp          0x40007ffe10 0x40007ffe10
s1          0x0      0
a0          0x0      0
a1          0x5      5
a2          0x0      0
a3          0x0      0
a4          0x20     32
a5          0x1      1
```

在汇编程序的调试中，追踪寄存器的值是很实用的。

3.3 提交要求

请将实验报告命名为：学号_姓名_实验 4.docx（或者 pdf），然后和通过了测试的源码文件 `mul.s`、`fac.s` 一起打包，压缩包的名字为：学号_姓名_实验 4.zip 然后将压缩包通过邮件提交。

附录：实验报告模板

RISC-V 汇编程序编程练习

1、实验目的

通过编写和运行 RISC-V 汇编程序，熟悉 RISC-V 基本指令集中的指令、RISC-V 汇编的编程规范，了解递归函数的调用过程。

2、实验步骤（包括实验结果，数据记录、截图等）

- （1）使用 RV64I 指令集编写 32 位整数乘法函数 `mymul`
- （2）使用 RV64I 指令集和 `mymul` 函数实现自然数阶乘函数 `myfac`
- （3）编译和测试结果

3、实验分析和总结

4、实验收获、存在问题、改进措施或建议等